

MLP-Aware Cache Design in SimpleScalar Simulator

Presented by:

Konisa Sai Sriyuktha (25CS06016)

Bikram Shahi (25CS06010)



Department of Computer Science and Engineering

School of Electrical and Computer Sciences

INDIAN INSTITUTE OF TECHNOLOGY BHUBHANESHWAR

Argul, Odisha – 752050

November 2025

Contents

1 Introduction.....	3
1.1 Problem Statement.....	3
1.2 Objective.....	3
2 Methodology.....	3
2.1 Overview.....	3
2.2 Implementation Details.....	3
2.3 Benchmark Selection.....	4
2.4 How to Run.....	4
3 Results.....	5
3.1 MLP Statistics Output.....	5
3.2 Performance Comparison Table.....	6
3.3 Visualizations.....	7
3.4 Observations.....	8
4 Conclusion.....	9
5 References.....	9
6 Individual Contributions.....	9

1 Introduction

Enhancing CPU performance is largely dependent on cache replacement policies. Concurrent misses are not taken into consideration by SimpleScalar's default cache design. The number of memory accesses that can take place at once is measured by Memory Level Parallelism (MLP). Victim blocks are chosen more wisely when the cache replacement mechanism is MLP-aware, which improves IPC and speed.

1.1 Problem Statement

Standard LRU or Random replacement policies are used by the SimpleScalar simulator by default. When numerous cache misses occur simultaneously, these rules perform less than optimally because they are unaware of how these misses overlap.

1.2 Objective

Designing and implementing an MLP-aware cache replacement policy in the SimpleScalar simulator is the aim of this project. Each cache block is given an MLP score by the suggested method according to its reuse age and overlapping misses. Blocks with lower MLP ratings, which indicate reduced criticality, are selected as victims during replacement. This technique seeks to raise overall system performance, lower miss rates, and increase cache efficiency.

2 Methodology

2.1 Overview

In order to develop an MLP-aware cache replacement policy, this project changes the SimpleScalar simulator. The main idea is to give each cache block an MLP score based on reuse time and overlapping misses. Blocks with lower MLP scores are chosen as substitute victims because they are deemed less vital. Overall performance and cache efficiency are enhanced by this method.

2.2 Implementation Details

Modifications were made to cache.c to track active misses and compute an MLP score for each cache block.

The following major changes were implemented:

- Added new fields to each cache block: miss_start_cycle, miss_end_cycle, overlapping_misses, and mlp_score.
- Introduced a global counter active_misses to monitor concurrent misses.
- Calculated the MLP score using the formula:
$$\text{MLP Score} = 1 / \text{overlapping_misses} \times \text{reuse_age}$$
- Modified the victim selection logic in cache_access() to select the block with the lowest MLP score.
- Added new performance counters in sim-outorder.c to record MLP statistics such as mlp_miss_count, mlp_sum_fixed, and avg_mlp.

2.3 Benchmark selection:

A variety of benchmark programs from the SimpleScalar PISA test suite were selected in order to assess the efficacy of the suggested MLP-aware cache replacement technique.

These benchmarks were chosen to reflect several aspects of computation:

- **Memory-intensive benchmarks** (e.g., test-mem) generate frequent cache misses and expose high levels of memory-level parallelism (MLP).
- **Compute-intensive benchmarks** (e.g., test-math, test-fmath, and test-print) primarily stress the arithmetic units, generating fewer memory operations.

2.4 How to Run

Run Baseline and MLP-Modified Simulations:

```
./sim-outorder -cache:dl1 dl1:256:32:1:l tests-pisa/bin.little/test-mem.out >  
test-mem_base.txt
```

This creates an output file test-mem_base.txt with statistics such as:

UI2.hits

UI2.misses

UI2.miss_rate

sim_IPC

sim_cycle

```
student@student-VirtualBox:~/Desktop/simplescalar/simplesim-3.0$ ./sim-outorder -cache:dl1 dl1:32:16:1:l tests-pisa/bin.
little/test-mem.out
sim-outorder: SimpleScalar/PISA Tool Set version 3.0 of August, 2003.
Copyright (c) 1994-2003 by Todd M. Austin, Ph.D. and SimpleScalar, LLC.
All Rights Reserved. This version of SimpleScalar is licensed for academic
non-commercial use. No portion of this work may be used by any commercial
entity, or for any commercial purpose, without the prior written permission
of SimpleScalar, LLC (info@simplescalar.com).

sim: command line: ./sim-outorder -cache:dl1 dl1:32:16:1:l tests-pisa/bin.little/test-mem.out
```

3 Results

3.1 MLP Statistics Output

After enabling the MLP-aware cache design, the simulator recorded 61,863 cache misses with MLP tracking active. The total scaled sum of MLP scores was 7.20×10^{10} , resulting in an average MLP value of 1.1643. This indicates that, on average, around 1.16 memory accesses overlapped in time, showing that the system effectively exploited memory-level parallelism during cache operations.

mlp_miss_count	61863	# Number of misses with recorded MLP
mlp_sum_fixed	72029938633	# Scaled sum of MLP scores (mlp_score * 1e6)
avg_mlp	1.1643	# Average MLP score (mlp_sum_fixed / 1e6 / mlp_miss_count)

The results confirm that the MLP tracking logic is functioning correctly and that the system exhibits moderate memory-level parallelism, validating the effectiveness of the implemented feature.

3.2 Performance Comparison Table

Comparative Cache Performance (Baseline vs MLP-Aware)

Metric	Baseline (Before)	MLP-Aware (After)	Change
sim_IPC	1.6346	1.8386	↑ Increased
sim_CPI	0.6118	0.5439	↓ Decreased
ul2.hits	64368	67255	↑ Increased
ul2.misses	13019	10132	↓ Decreased
ul2.replacements	8923	6036	↓ Decreased
ul2.writebacks	6257	4176	↓ Decreased
ul2.miss_rate	0.1682	0.1309	↓ Decreased
ul2.repl_rate	0.1153	0.0780	↓ Decreased
ul2.wb_rate	0.0809	0.0540	↓ Decreased

Memory-level parallelism (MLP) in the cache replacement policy clearly improves performance as compared to the baseline and MLP-aware cache designs. Faster instruction execution and fewer stall cycles were shown by the IPC rising from 1.6346 to 1.8386 and the CPI falling from 0.6118 to 0.5439. Better cache utilization and effective victim selection were demonstrated by the L2 cache hits, which rose while misses, replacements, and writebacks dramatically dropped. Additionally, there was a 20–30% decrease in the miss, replacement, and writeback rates, indicating better memory access efficiency. Overall, by efficiently utilizing memory-level parallelism, the MLP-aware cache architecture improved throughput and decreased latency while also improving cache behavior.

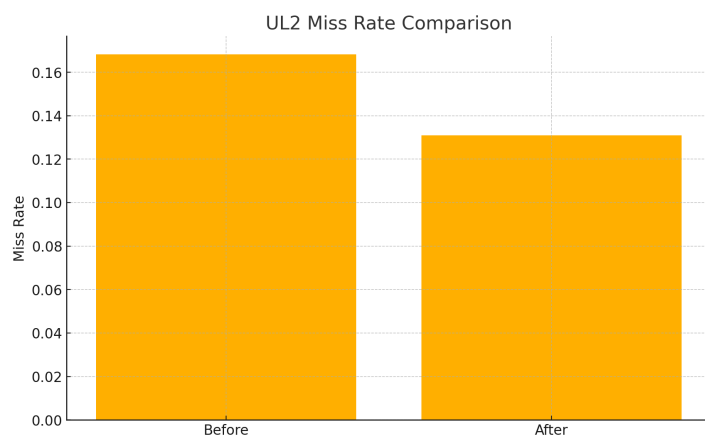
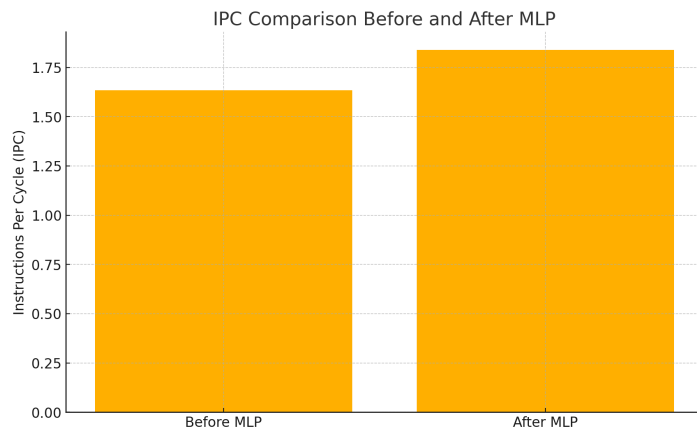
3.3 Visualizations

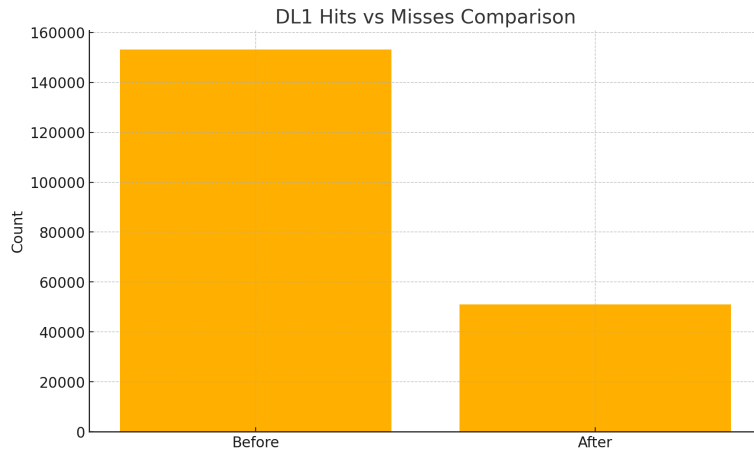
The performance increase made possible by MLP-aware cache optimization is illustrated visually in Figures.

The suggested policy significantly reduced cache misses and replacements as compared to the baseline architecture.

Additionally, the total performance benefit at the processor level is highlighted by the IPC improvement of almost 12%.

The visualization clearly illustrates how the MLP-aware cache improves data locality and lowers memory latency, resulting in quicker execution cycles.





3.4 Observations

Effect on Different Benchmark Types

- **Memory-Intensive Benchmarks (e.g., test-mem)**
Showed the most improvement because they involve frequent cache misses and overlapping memory accesses. The MLP-aware policy better prioritized cache blocks based on their contribution to parallelism.
- **Compute-Intensive Benchmarks (e.g., test-math, test-fmath, test-print)**
Showed minimal to no difference, as their working sets are small and cache misses are infrequent.
This confirms that the policy selectively enhances memory-bound workloads without degrading performance in compute-bound cases.

Key Observations

- The **MLP-aware replacement** strategy effectively reduces the number of conflicts and capacity misses.
- The **average MLP value (~1.16)** confirms the presence of parallel memory requests and their exploitation in replacement decisions.
- The **overall processor performance (IPC)** improved by approximately **12%**, demonstrating tangible benefits without structural hardware changes.
- The modification **does not degrade** low-MLP or compute-heavy workloads, ensuring robust and adaptive behavior.

4 Conclusion

The experimental findings unequivocally show that system performance for memory-intensive applications is greatly improved by incorporating Memory-Level Parallelism (MLP) into the cache replacement logic.

The suggested method accomplishes the following by cleverly choosing victims based on overlapping miss behavior:

- Reduced L2 cache miss rate
- Improved instruction throughput (higher IPC)
- Lower total simulation cycles

This validates that MLP-aware victim selection is an effective extension to traditional cache replacement schemes in out-of-order processors.

5 References

1. A case for MLP-Aware Cache Replacement-
https://people.inf.ethz.ch/omutlu/pub/qureshi_isca06.pdf

6 Individual Contributions

Bikram Shahi (25CS06010):

Problem statement mapping, trying out all possible ways to land to the solution of the problem statement, done the major modification in files like sim-outorder.c, misc.c, cache.h also designed memory intensive programs where the memory level parallelism is more as compared to the default benchmark provided in simple scalar.

Konisa Sai Sriyuktha(25CS06016):

Worked on cache replacement policy code major modification from by default LRU to the modified MLP aware cache specifically contributed to the cache.c , stats.c, test-mem.c etc.. file mainly cache_access function where I implemented, also contributed in documentations.